

$$Q, K, V \in \mathbb{R}^{d \times d}$$

$$\text{Input } X \in \mathbb{R}^{n \times d}$$

$$Q = QX \in \mathbb{R}^{n \times d}$$

$$K = KX \in \mathbb{R}^{n \times d}$$

$$V = VX \in \mathbb{R}^{n \times d}$$

$$A = QK^T \in \mathbb{R}^{n \times n}$$

$$\text{output} = \begin{bmatrix} -a_1- \\ -a_2- \\ \vdots \\ -a_n- \end{bmatrix}_{n \times n} \begin{bmatrix} -v_1- \\ -v_2- \\ \vdots \\ -v_n- \end{bmatrix}_{n \times d}$$

$A \qquad V$

In continual learning, we have buffer size = n ,
and input data on which we want prediction
 $\therefore$ Total input $n+1$

$$\text{output} = \begin{bmatrix} -a_1- \\ -a_2- \\ \vdots \\ -a_n- \\ -a_{n+1}- \end{bmatrix} \begin{bmatrix} -v_1- \\ \vdots \\ -v_{n+1}- \end{bmatrix}_{(n+1) \times d}$$

But in continual learning we only want
predictⁿ for $n+1$ datapoint given data in
buffer

∴ We can have attention to focus only on $n+1$ datapoint instead of all datapoints.
Simplifying this and taking transpose we get

$$\begin{bmatrix} 1 \\ v_1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ v_2 \\ 1 \end{bmatrix} \dots \begin{bmatrix} 1 \\ v_{n+1} \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ a_{\text{last}} \\ 1 \end{bmatrix}, a_{\text{last}} = a_{n+1}$$

$d \times (n+1) \quad (n+1) \times 1$

⇒ This is a linear combination of value vectors
~~and~~

$$a_{\text{last}}^1 \begin{bmatrix} 1 \\ v_1 \\ 1 \end{bmatrix} + a_{\text{last}}^2 \begin{bmatrix} 1 \\ v_2 \\ 1 \end{bmatrix} + \dots + a_{\text{last}}^{n+1} \begin{bmatrix} 1 \\ v_{n+1} \\ 1 \end{bmatrix}$$

Here, $a_{\text{last}}^i \Rightarrow$ Relationship of input $n+1$ with all the datapoints 1 to n

$$a_{\text{last}}^i = \underbrace{x_{n+1}}_{\text{input}} \underbrace{Q_{\alpha} (x_i K_B)^T}_{\text{support}}$$

∴ a_{last}^i is the dot product of transformⁿ of input x_{n+1} and elements in buffer x_i / support

Instead we can simplify this by directly feeding input data and buffer through a learnt distance metric Θ and replaced d, B, r

$$\therefore a_{lost}^i = b_{\theta}(x_{n+1}, x_i)$$

Further, we observe empirically that value vector itself is unnecessary and we can have linear combination of one hot encoding itself. Hence, we can further simplify as,

$$b_{\theta}(x_{n+1}, x_1) \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} + b_{\theta}(x_{n+1}, x_2) \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} + \dots$$

The rationale is that we are taking a weighted sum of the support labels where weight is the relationship between input datapoint x_{n+1} and x_i support from the buffer.